

Tasca S3.01. Manipulación de tablas

Alumno: Jose Manuel Romero García

Curso: Especialidad Analista de datos

Mentor: Yunier Córdova

Fecha de entrega: 14/02/2026

El objetivo de esta tarea ha sido aplicar operaciones de manipulación de estructuras y datos en una base de datos relacional MySQL denominada **transactions**, utilizando principalmente la herramienta **MySQL Workbench**.

Se han realizado tres niveles progresivos de dificultad que incluyen:

- Creación y modificación de tablas.
- Definición de claves primarias y foráneas.
- Inserción, eliminación y actualización respetando la integridad referencial.
- Creación y consulta de vistas analíticas.
- Documentación gráfica mediante diagramas EER

Herramientas utilizadas:

- MySQL Workbench.
- Scripts de estructura y datos proporcionados por el profesorado.

1. NIVEL 1. Creación de tablas y relaciones.

1.1. Ejercicio 1. Creación de la tabla `credit_card` e inserción de datos.

```
#Creación de la tabla credit_card#  
  
CREATE TABLE IF NOT EXISTS credit_card(  
  id VARCHAR(15) PRIMARY KEY,  
  iban VARCHAR(50),  
  pan VARCHAR(25),  
  pin VARCHAR(4),  
  cvv INT,  
  expiring_date VARCHAR(100)  
);  
  
#Procedo a abrir el documento de los datos de credit_card del documento datos_introducir_sprint3_credit y hago la comprobación#  
  
SELECT *  
FROM credit_card;  
  
#Creación de la FK en la tabla Transaction (credit_card_id) para relacionarse con Credit_card#
```

id	iban	pin	cvv	expiring_date	fecha_actual
1	XXXXXX XXXX XXXX XXXX XXXX	1234	5678	11/26/26	2026
2	XXXXXX XXXX XXXX XXXX XXXX	5678	9012	11/26/26	2026
3	XXXXXX XXXX XXXX XXXX XXXX	9012	3456	11/26/26	2026
4	XXXXXX XXXX XXXX XXXX XXXX	3456	7890	11/26/26	2026

Action Output

Time	Action	Response
17:11:11	SELECT * FROM credit_card	5001 row(s) returned

Para la creación de la tabla `credit_card`, sigo la siguiente estructura y una vez creada, procedo a la inserción de los datos, introduzco los datos del documento "dades_introducir_credit":

Para crear la tabla `Credit_card` he tenido en seguido los campos que aparecen en el documento y me he basado en las otras tablas ya creadas para ver el tipo de datos que era necesario asociarle. Los campos y los tipos de datos son: `id`, `VARCHAR(15)` que también es PK; `iban`, `VARCHAR(50)`; `pan`, `VARCHAR(25)`; `pin`, `VARCHAR(4)`; `ccv`, `INT`; `expiring_date`, `VARCHAR(100)`.

Justificación de tipos de datos:

- `id VARCHAR(15)` → identificador alfanumérico con formato fijo
- `expiring_date VARCHAR(100)` → permite flexibilidad en diferentes formatos de fecha de caducidad
- `ccv INT` → valor numérico sin ceros a la izquierda

Una vez creada la **tabla `credit_card`**, procedo a crear la relación que se establece con la **tabla `transaction`**, siendo **`transaction` la tabla principal**. La relación es de **1 a N**. Siendo el campo `id` la **PK** de la **tabla `credit_card`**, que se relaciona con el campo `credit_card_id` de la **tabla `transaction`**, **FK**.

Creación de la FK en la tabla `transaction`:

```
23
24 • ALTER TABLE transaction
25   ADD CONSTRAINT fk_credit_card_id
26   FOREIGN KEY (credit_card_id)
27   REFERENCES credit_card(id);
28
29
30
31
```

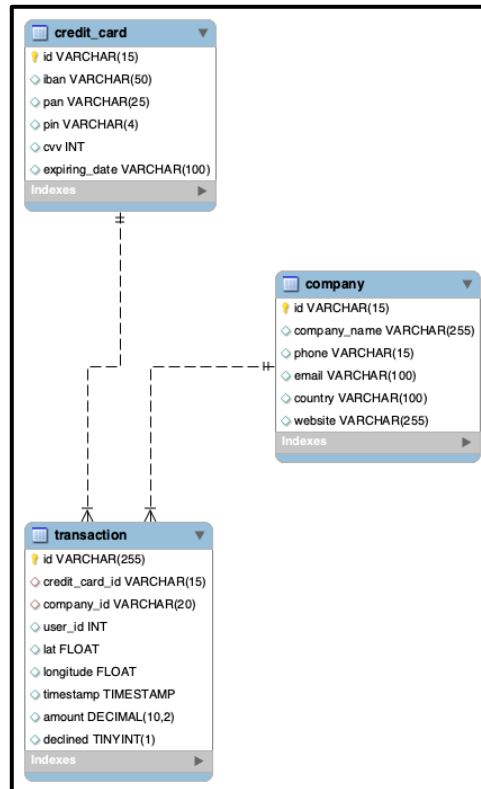
100% 1:30

Action Output

	Time	Action	Response
✓	12:48:50	ALTER TABLE transaction ADD CONSTRAINT fk_credit_card_id FOREIGN KEY (credit_card_id) REFER...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
✓	12:53:33	ALTER TABLE transaction DROP FOREIGN KEY fk_credit_card	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
✓	12:54:05	ALTER TABLE transaction ADD CONSTRAINT fk_credit_card_id FOREIGN KEY (credit_card_id) REF...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Diagrama resultante

El diagrama resultante lo obtengo directamente de MySQL, es el siguiente:



El diagrama resultante nos muestra las relaciones que se establecen entre las tablas secundarias (**credit_card** y **company**) y la tabla principal (**transaction**). Las relaciones son de **1 a N**. La **tabla company** tiene como **PK** el campo **id**, la **tabla credit_card** tiene como **PK** el campo **id**. Estos campos corresponderían con las **FK** que encontramos en la **tabla transaction**, que tiene como **PK** el campo **id**, y como **FK** tiene **company_id**, **credit_card_id**.

1.2. Ejercicio 2. Identificación de un error y proceder al cambio de su valor.

Nos comunican que hay un error en un dato, en la tarjeta de crédito con ID: CcU-2938. El valor que debe mostrar es el siguiente: TR32345631221357681769999

Antes del cambio:

36

37 • SELECT iban

38 FROM credit_card

39 WHERE id = 'CcU-2938';

40

41

0%

17:38

Result Grid

Filter Rows: Search

Export:

iban

TR301950312213576817638661

credit_card 29

Action Output

Time

Action

Response

1

12:59:06

SELECT iban FROM credit_card WHERE id = 'CcU-2938'

1 row(s) returned

Realización del cambio mediante update:

42

43 • UPDATE credit_card

44 SET iban = 'TR323456312213576817699999'

45 WHERE id = 'CcU-2938';

46

47

48

49

50

51

52

53

54

55

56

100%

1:48

Action Output

Time

Action

Response

1

13:00:12

UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938'

1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

Comprobación del resultado:

48 • SELECT iban

49 FROM credit_card

50 WHERE id = 'CcU-2938';

51

52

100%

17:49

Result Grid

Filter Rows: Search

Export:

iban

TR323456312213576817699999

credit_card 30

Action Output

Time

Action

Response

1

13:01:05

SELECT iban FROM credit_card WHERE id = 'CcU-2938'

1 row(s) returned

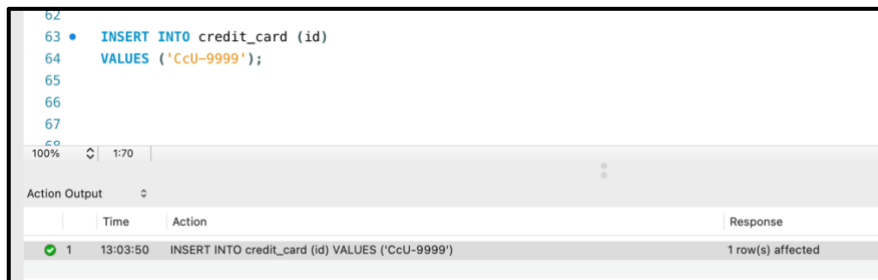
1.3. Ejercicio 3. Insertar una nueva transacción.

Al intentar realizar la introducción de los datos mediante MySQL no me permite hacerlo porque tengo dos **FK** entre los datos que debo introducir.

Tengo que introducir los datos que son **FK** primero en las otras tablas (**credit_card** y **company**) donde son **PK**.

Introducción del dato en la **tabla credit_card**:

```
62
63 • INSERT INTO credit_card (id)
64   VALUES ('Ccu-9999');
65
66
67
```



	Time	Action	Response
1	13:03:50	INSERT INTO credit_card (id) VALUES ('Ccu-9999')	1 row(s) affected

Introducción del dato en la **tabla company**:

```
67
68 • INSERT INTO company (id)
69   VALUES ('b-9999');
70
71
72
73
74
75
76
77
78
```



	Time	Action	Response
1	13:04:55	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected

Ahora me permite hacer el ingreso de los nuevos datos en la **tabla transaction**:

```
58 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
59   VALUES ('10081D1D-5823-A76C-55EF-C568E49A990D', 'Ccu-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);
60
61 #Haig d'inssertar primer aquests valors a les altres taules#
62
63 • INSERT INTO credit_card (id)
64   VALUES ('Ccu-9999');
65
66
67 • INSERT INTO company (id)
68   VALUES ('b-9999');
69
70
71
72
73
74
75
76
77
78
```



	Time	Action	Response
1	13:05:46	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('10081D1D-5823-A76C-55EF-C568E49A990D', 'Ccu-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);	1 row(s) affected

Comprobación:

```
72 #Comprobació#
73
74 • SELECT *
75 FROM transaction
76 WHERE credit_card_id = 'CcU-9999';
77
78
```

00% 17:75

Result Grid Filter Rows: Search Edit: Export/Import:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	INDEX	111.11	0
INDEX	INDEX	INDEX	INDEX	INDEX	INDEX	INDEX	INDEX	INDEX

transaction 31

Action Output

	Time	Action	Response
1	13:09:59	SELECT * FROM transaction WHERE credit_card_id = 'CcU-9999'	1 row(s) returned

1.4. Ejercicio 4. Eliminación de la columna pan

El ejercicio pide que eliminemos la columna pan, esto se hace mediante un drop y posteriormente, hago la comprobación:

```
85 • ALTER TABLE credit_card
86 DROP COLUMN pan;
87
88
89
90 • DESC credit_card;
91
92
93
```

100% 1:88

Result Grid Filter Rows: Search Export:

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PK		
iban	varchar(50)	YES			
pin	varchar(4)	YES			
cvv	int	YES			
expiring_date	varchar(100)	YES			

Result 32

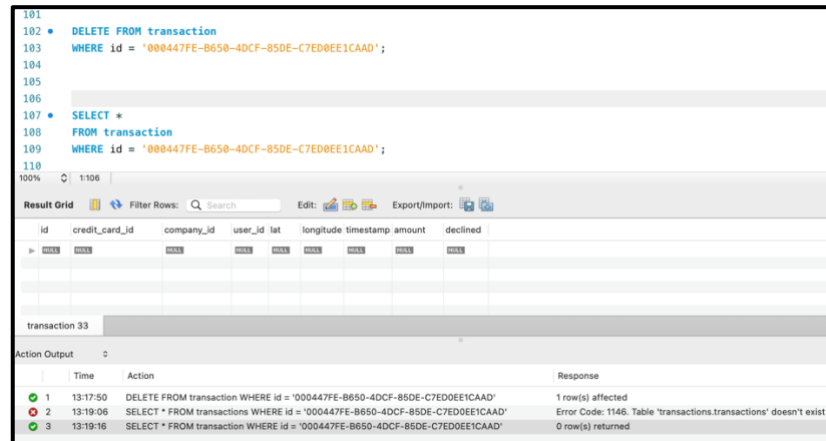
Action Output

	Time	Action	Response
1	13:09:59	SELECT * FROM transaction WHERE credit_card_id = 'CcU-9999'	1 row(s) returned
2	13:11:45	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
3	13:12:06	DESC credit_card	5 row(s) returned

2. NIVEL 2. Eliminar una entrada, creación de vistas y consulta mediante filtrado.

2.1. Ejercicio 1. Eliminación de una entrada.

El ejercicio pide eliminar la entrada id = 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la **tabla transaction**. Hago un DELETE y después un SELECT para comprobar el resultado:



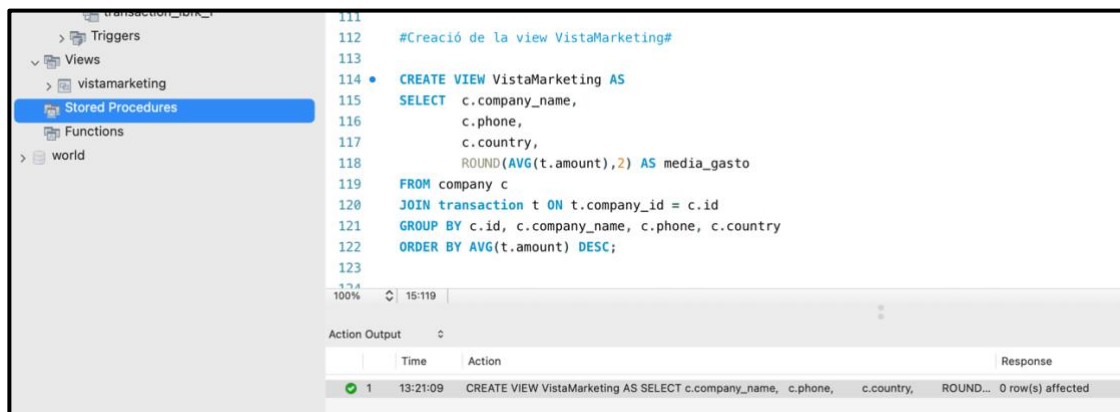
```
101
102 • DELETE FROM transaction
103 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
104
105
106
107 • SELECT *
108 FROM transaction
109 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
110
```

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
transaction 33								

Time	Action	Response
13:17:50	DELETE FROM transaction WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected
13:19:06	SELECT * FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	Error Code: 1146. Table 'transactions.transactions' doesn't exist
13:19:16	SELECT * FROM transaction WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	0 row(s) returned

2.2. Ejercicio 2. Creación de una vista (view)

EL ejercicio nos pide la creación de la VIEW VistaMarketing:



```
111
112 #Creació de la view VistaMarketing#
113
114 • CREATE VIEW VistaMarketing AS
115 SELECT c.company_name,
116        c.phone,
117        c.country,
118        ROUND(AVG(t.amount),2) AS media_gasto
119 FROM company c
120 JOIN transaction t ON t.company_id = c.id
121 GROUP BY c.id, c.company_name, c.phone, c.country
122 ORDER BY AVG(t.amount) DESC;
123
```

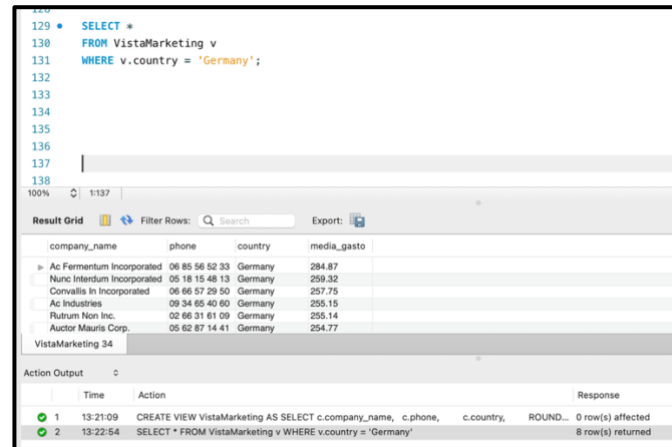
Time	Action	Response
13:21:09	CREATE VIEW VistaMarketing AS SELECT c.company_name, c.phone, c.country, ROUND...	0 row(s) affected

Para crear la View utilizo en el **SELECT, ROUND(AVG(t.amount))**, pero para ordenar utilizo **AVG(t.amount)** para evitar que los redondeos me presenten datos erróneos si tengo dos compañías con un **AVG(t.amount)** que al redondear quede igual. Además, en el **GROUP BY** incluyo **c.id** (el **id** de la **tabla company**), ya que este identificador es

único para cada compañía, de esta forma evito problemas si tengo dos compañías con el mismo nombre.

2.3. Ejercicio 3. Una vez creada la vista, hacemos una consulta con filtrado.

El ejercicio nos pide que una vez hemos creado la vista **VistaMarketing**, hagamos una consulta sobre las empresas que están localizadas en Alemania **country = 'Germany'**:



The screenshot shows a SQL query editor with the following query:

```
129 • SELECT *
130 FROM VistaMarketing v
131 WHERE v.country = 'Germany';
132
133
134
135
136
137
138
```

Below the query, the 'Result Grid' shows the results of the query. The columns are: company_name, phone, country, media_gasto.

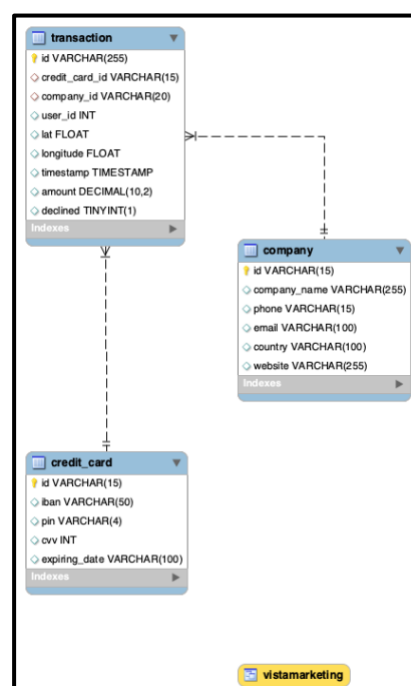
company_name	phone	country	media_gasto
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87
Nunc Interdum Incorporated	05 18 15 48 13	Germany	259.32
Convalis In Incorporated	06 66 57 29 50	Germany	257.75
Ac Industries	09 34 65 40 80	Germany	255.15
Rutrum Non Inc.	02 66 31 61 09	Germany	255.14
Auctor Mauris Corp.	05 62 87 14 41	Germany	254.77

Below the result grid, the 'Action Output' shows the execution of the query.

Time	Action	Response
13:21:09	CREATE VIEW VistaMarketing AS SELECT c.company_name, c.phone, c.country, ROUND...	0 row(s) affected
13:22:54	SELECT * FROM VistaMarketing v WHERE v.country = 'Germany'	8 row(s) returned

Diagrama obtenido al final del Nivel 2

Al final del nivel 2 tenemos el siguiente diagrama:



3. Nivel 3. Modificaciones avanzadas y nueva vista analítica

3.1. Ejercicio 1. Explicación detallada de los pasos a seguir para obtener el diagrama resultante en este nivel.

Pasos que se han seguido para llegar al diagrama del Nivel 3:

- Creación de la tabla user:

He cargado los datos del documento **“estructura_dades_user”**, para tener la estructura de la **tabla user**. Insertar **tabla user**:

```
1 * CREATE TABLE IF NOT EXISTS user (  
2   id CHAR(10) PRIMARY KEY,  
3   name VARCHAR(100),  
4   surname VARCHAR(100),  
5   phone VARCHAR(150),  
6   email VARCHAR(150),  
7   birth_date VARCHAR(100),  
8   country VARCHAR(150),  
9   city VARCHAR(150),  
10  postal_code VARCHAR(100),  
11  address VARCHAR(255)  
12 );  
13
```

	Time	Action	Response
1	13:21:09	CREATE VIEW VistaMarketing AS SELECT c.company_name, c.phone, c.country, ROUND...	0 row(s) affected
2	13:22:54	SELECT * FROM VistaMarketing v WHERE v.country = 'Germany'	8 row(s) returned
3	13:49:59	CREATE TABLE IF NOT EXISTS user (id CHAR(10) PRIMARY KEY, name VARCHAR(100), surname...	0 row(s) affected

- He cargado el documento **"dades_introduir_user"**, para introducir los datos de la **tabla user**.

```
5 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
7 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
8 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
9 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
10 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
11 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
12 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
13 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
14 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
15 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
16 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
17 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
18 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
19 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
20 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
21 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
22 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
23 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
24 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
25 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
26 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
27 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add  
28 * INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, add
```

	Time	Action	Response
50...	13:51:37	INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, addres...	1 row(s) affected
50...	13:51:37	INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, addres...	1 row(s) affected
50...	13:51:37	INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, addres...	1 row(s) affected

- La **tabla credit_card**, se había creado anteriormente, siguiendo el modelo de las tablas anteriores.
- Una vez creada la **tabla credit_card** se habían añadido los datos del documento "**dades_introduir_credit**".
- La **tabla credit_card** tiene como **PK** el campo (**id**). Esto queda reflejado en la creación de la tabla. Se había creado la FK de la tabla transaction con la tabla credit_card.
- Lo siguiente que se ha hecho ha sido crear la **FK** de la **tabla transaction** que se relaciona con la **tabla user**. Para ello primero tenemos que hacer que los campos **transaction.user_id** y **user.id** tengan el mismo tipo de dato. En **transaction** el tipo de dato es **INT** y en **user** el tipo de dato es un **CHAR(10)**. Para ello modificamos el tipo de dato del campo **id** de la **tabla user** a **INT**.

Tipo de dato de id en la **tabla user**, vemos que **id** es un **CHAR(10)**:

Field	Type	Null	Key	Default	Extra
id	char(10)	NO	PRI		
name	varchar(100)	YES			
surname	varchar(100)	YES			
phone	varchar(150)	YES			
email	varchar(150)	YES			
birth_date	varchar(100)	YES			

Time	Action	Response
50... 13:51:37	INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, address)	1 row(s) affected
50... 13:58:38	DESC user	10 row(s) returned
50... 13:58:58	DESC transaction	9 row(s) returned

Tipo de dato en la **tabla transaction**, vemos que **user_id** es un **INT**:

Field	Type	Null	Key	Default	Extra
id	varchar(255)	NO	PRI		
credit_card_id	varchar(15)	YES	MUL		
company_id	varchar(20)	YES	MUL		
user_id	int	YES			
lat	float	YES			
longitude	float	YES			

Time	Action	Response
50... 13:51:37	INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, address)	1 row(s) affected
50... 13:58:38	DESC user	10 row(s) returned
50... 13:58:58	DESC transaction	9 row(s) returned

Se realiza el cambio del dato de **id** en la **tabla user**:

```
141 #Cambio user(id) de char(10) a int para poder convertir transaction(user_id) en FK
142
143 • ALTER TABLE user
144   MODIFY COLUMN id INT;
145
146
147
148
149
150
151
152
153
154
155
```

100% 1:148

Action Output

	Time	Action	Response
50...	13:58:38	DESC user	10 row(s) returned
50...	13:58:58	DESC transaction	9 row(s) returned
50...	14:06:55	ALTER TABLE user MODIFY COLUMN id INT	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

- Comprobación del cambio de tipo de dato en la **tabla user**:

```
147
148 #Comprobación del cambio realizado#
149
150 • DESC user;
151
152
```

100% 37:148

Result Grid

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI		
name	varchar(100)	YES			
surname	varchar(100)	YES			
phone	varchar(150)	YES			
email	varchar(150)	YES			
birth_date	varchar(100)	YES			

Result 35

Action Output

	Time	Action	Response
50...	13:58:58	DESC transaction	9 row(s) returned
50...	14:06:55	ALTER TABLE user MODIFY COLUMN id INT	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
50...	14:14:54	DESC user	10 row(s) returned

- Creación de la **FK** en la **tabla credit_card** que se relaciona con la **tabla user**, primero he tenido que introducir el dato que le faltaba a la **tabla user**, este dato viene especificado en el Ejercicio 3 del Nivel 1 :

```
161
162 • INSERT INTO user (id)
163   VALUES ('9999');
164
165
166 #Al introducir este dato ya puedo hacer la FK#
167
168
169
```

100% 1:169

Action Output

	Time	Action	Response
1	14:22:19	INSERT INTO user (id) VALUES ('9999')	1 row(s) affected

- Ahora ya se puede establecer la relación de la **FK** en la **tabla transaction**, ya que el dato introducido anteriormente es **PK** en la **tabla user**, al tratarse de un dato que es **PK** este valor no puede ser nulo:

```

155 • ALTER TABLE transaction
156 ADD CONSTRAINT fk_transaction_user_id
157 FOREIGN KEY (user_id)
158 REFERENCES user(id);
159
160 #Al hacerlo da error porque no falta introducir un dato en la tabla user#
161
162 • INSERT INTO user (id)
163 VALUES ('9999');
164
165
166 #Al introducir este dato ya puedo hacer la FK#
167
168
169

```

100% 38-156

	Time	Action	Response
1	14:22:19	INSERT INTO user (id) VALUES ('9999')	1 row(s) affected
2	14:24:29	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_user_id FOREIGN KEY (user_id) REF...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

- Se ha eliminado la **columna website** de la **tabla company**.

```

168 #El.Liminació columna website de taula Company#
169
170 • ALTER TABLE company
171 DROP COLUMN website;
172
173 #COMPROBACIÓ#
174
175 • DESC company;
176

```

100% 21-171

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI		
company_name	varchar(255)	YES			
phone	varchar(15)	YES			
email	varchar(100)	YES			
country	varchar(100)	YES			

Result 36

	Time	Action	Response
1	14:24:29	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_user_id FOREIGN KEY (user_id) REF...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
2	14:24:29	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_user_id FOREIGN KEY (user_id) REF...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
3	14:36:16	ALTER TABLE company DROP COLUMN website	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
4	14:36:43	DESC company	5 row(s) returned

- Se ha cambiado el nombre del campo **email** a **personal_email** en la **tabla user**.

```

180 • ALTER TABLE user
181 RENAME COLUMN email to personal_email;
182
183 #Comprobació#
184
185 • DESC user;
186
187
188

```

100% 11-185

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI		
name	varchar(100)	YES			
surname	varchar(100)	YES			
phone	varchar(150)	YES			
personal_email	varchar(150)	YES			
birth_date	varchar(100)	YES			

Result 37

	Time	Action	Response
1	14:36:43	DESC company	5 row(s) returned
2	14:36:43	DESC company	5 row(s) returned
3	14:38:33	ALTER TABLE user RENAME COLUMN email to personal_email	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
4	14:38:40	DESC user	10 row(s) returned

- Se ha cambiado el nombre de la **tabla user** a **data_user**.

```

188
189 • RENAME TABLE user to data_user;
190
191 #Comprobació#
192
193 • DESC data_user;
194
195 #Cambiar tipus de dato a credit_card_id de VARCHAR(15) a VARCHAR(20) a la taula transaction#
196
100% 15:191

```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI		
name	varchar(100)	YES			
surname	varchar(100)	YES			
phone	varchar(150)	YES			
personal_email	varchar(150)	YES			
birth_date	varchar(100)	YES			

Result 38

Time	Action	Response
14:38:40	DESC user	10 row(s) returned
14:41:48	RENAME TABLE user to data_user	0 row(s) affected
14:42:01	DESC data_user	10 row(s) returned

- Se ha cambiado el tipo de dato de **credit_card_id** en la **tabla transaction** de VARCHAR(15) a VARCHAR(20). Según se ve en el diagrama aportado.

```

197 • ALTER TABLE transaction
198   MODIFY COLUMN credit_card_id VARCHAR(20);
199
200 #Comprobació#
201
202 • DESC transaction;
203
100% 1:199

```

Field	Type	Null	Key	Default	Extra
id	varchar(255)	NO	PRI		
credit_card_id	varchar(20)	YES	MUL		
company_id	varchar(20)	YES	MUL		
user_id	int	YES	MUL		
lat	float	YES			
longitude	float	YES			

Result 39

Time	Action	Response
14:42:01	DESC data_user	10 row(s) returned
14:44:37	ALTER TABLE transaction MODIFY COLUMN credit_card_id VARCHAR(20)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
14:44:13	DESC transaction	9 row(s) returned

- Se ha cambiado el tipo de dato del campo **id** de la **tabla credit_card** de VARCHAR(15) a VARCHAR(20). Según se ve en el diagrama aportado.

```

205
206 • ALTER TABLE credit_card
207   MODIFY COLUMN id VARCHAR(20);
208
209 #Comprobació#
210
211 • DESC credit_card;
212
100% 15:209

```

Field	Type	Null	Key	Default	Extra
id	varchar(20)	NO	PRI		
iban	varchar(50)	YES			
pin	varchar(4)	YES			
cvv	int	YES			
expiring_date	varchar(100)	YES			

Result 41

Time	Action	Response
14:47:47	DESC credit_card	5 row(s) returned

- Creación de la **columna fecha_actual** con el tipo de dato **date** y aceptación de **NULL**.

```

215 • ALTER TABLE credit_card
216   ADD COLUMN fecha_actual date NULL;
217
218 #Comprobación#
219
220 • DESC credit_card;
221

```

100% 24:215

Result Grid Filter Rows: Search Export:

Field	Type	Null	Key	Default	Extra
id	varchar(20)	NO	PK		
iban	varchar(50)	YES			
pin	varchar(4)	YES			
cvv	int	YES			
expiring_date	varchar(100)	YES			
fecha_actual	date	YES			

Result 42

Action Output 0

	Time	Action	Response
1	14:49:01	ALTER TABLE credit_card ADD COLUMN fecha_actual date NULL	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
2	14:49:08	DESC credit_card	6 row(s) returned

- Se ha cambiado el tipo de dato de **expiring_date** en la **tabla transaction** de **VARCHAR(100)** a **VARCHAR(20)**, según se ve en el diagrama aportado:

```

224 • ALTER TABLE credit_card
225   MODIFY COLUMN expiring_date VARCHAR(20);
226
227 #Comprobación#
228
229 • DESC credit_card;
230
231

```

100% 1:230

Result Grid Filter Rows: Search Export:

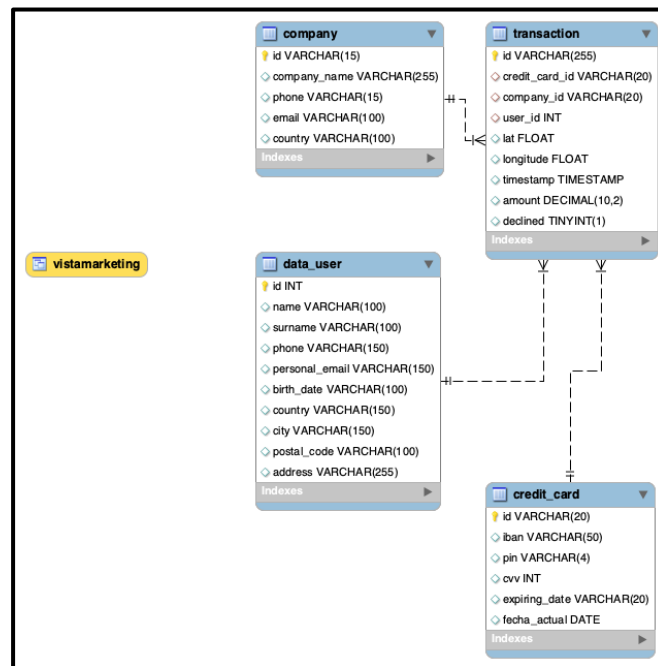
Field	Type	Null	Key	Default	Extra
id	varchar(20)	NO	PK		
iban	varchar(50)	YES			
pin	varchar(4)	YES			
cvv	int	YES			
expiring_date	varchar(20)	YES			
fecha_actual	date	YES			

Result 43

Action Output 0

	Time	Action	Response
1	14:55:38	ALTER TABLE credit_card MODIFY COLUMN expiring_date VARCHAR(20)	5001 row(s) affected Records: 5001 Duplicates: 0 Warnings: 0
2	14:56:22	DESC credit_card	6 row(s) returned

Nuevo diagrama con los cambios realizados en el Nivel 3:



3.2. Ejercicio 2. Creación de una nueva vista.

El ejercicio pide que creamos la vista **InformeTecnico**, que añadamos información relevante de las otras tablas y realicemos cambios de nombre a las columnas para mayor claridad:

```

SELECT t.id AS transaction_id,
       d.name AS customer_name,
       d.surname AS customer_surname,
       c.c.iban,
       c.c.company_name,
       c.c.id AS company_identity,
       d.country AS country_customer,
       c.c.country AS country_company,
       t.amount
FROM transaction t
JOIN data_user d ON d.id = t.user_id
JOIN credit_card c_c ON c_c.id = t.credit_card_id
JOIN company c ON c.id = t.company_id
ORDER BY t.id DESC;

SELECT *
FROM InformeTecnico;

```

Result Grid

transaction_id	customer_name	customer_surname	iban	company_name	company_identity	country_customer	country_company	amount
FFFD31D6-9495-47CE-B54A-7D8BE1CC274B	Bnvgjl	Tprvms	XX794814451211289182490922	Turpia Company	b-246			
FFFC776D-ECF0-4985-A200-B2A7B75998FC	Dnied	Vlqjdl	XX636251701647892036676034	Anet Nulla Donec Corporation	b-226			
FFFC9E8D-27C7-4ADE-98F2-7533EF4DF126	Securp	Fachvrlj	XX162677143304223631437567	Nunc Interdum Incorporated	b-239			
FFFB27D0-F33A-4D5D-999B-E5307C53CC64	Gjgga	Urzgjn	XX395114267082019952567562	Viverra Donec Foundation	b-256			
FFFE9E3E-234E-408C-ABCF-F9CAD577224A	Yahmq	Zpsjpled	XX8845462156537570367941	Conwallis Incorporated	b-228			
FFFE178-6CD2-4DF9-9980-48AE06880981	Jevexp	Xwexzwmn	XX321405515711654384711481	Mus Aenean Eget Foundation	b-226			
FFFB67C9-17B5-4B1F-AFD9-F8023AAA449E	Fqngjd	Lvhfyxvi	XX278446342932680979729426	Cras Vehicula Aliquet Industries	b-255			

Action Output

	Time	Action	Response
5	15:02:52	DESC transaction	9 row(s) returned
6	15:14:00	CREATE VIEW InformeTecnico AS SELECT t.id AS transaction_id, d.name AS customer_name, ...	0 row(s) affected
7	15:14:40	SELECT * FROM InformeTecnico	100000 row(s) returned

Para la creación de esta **Vista InformeTecnico** he creído necesario incorporar algunos datos que me parecen importantes. Por una lado he añadido el **company_identity (c.id)**, por si acaso tengo dos compañías con el mismo nombre.

También he añadido tanto el **country_customer (d.country)**, como el **country_company (c.country)**, porque me parece interesante ver el origen del cliente y el origen de la compañía donde hace las transacciones. Como estamos hablando de clientes he cambiado los campos **name** y **surname** por **customer_name** y **customer_surname**.

También creo que es importante saber la cantidad que se ha gastado, por lo que añado **amount** a la vista. Finalmente, he cambiado el nombre del **id** de la **transaction** a **transaction_id** para que resulte fácilmente identificable.